

TP1 — Modèles, fenêtre de contexte, entités et tokens

Durée estimée : 2h30–3h **Niveau :** Débutant **Prérequis :** Claude Code installé, session ouverte dans un dossier de travail vide **Format :** Exercices guidés pas à pas — chaque prompt est fourni, copiez-collez et observez **Ce que vous allez produire :** un `CLAUDE.md`, une première skill, un `.claudeignore`

Mise en place

Créez un dossier de travail et ouvrez Claude Code dedans :

```
mkdir ~/atelier-claude && cd ~/atelier-claude
claude
```

Vous devez voir le prompt interactif de Claude Code. Gardez ce terminal ouvert pendant tout le TP.

Partie 1 — Les modèles en pratique

Avant de configurer quoi que ce soit, observez comment le choix du modèle change les réponses.

Exercice 1 — Même question, trois modèles

Contexte : Haiku, Sonnet et Opus sont trois niveaux du même modèle Claude. Ils ne donnent pas la même qualité de réponse pour le même coût.

Étape 1. Dans Claude Code, tapez exactement ce prompt :

```
Explique le pattern Observer en programmation en 5 lignes maximum, avec un exemple concret en Python.
```

Notez la réponse. Observez : combien de lignes ? Y a-t-il un exemple fonctionnel ?

Étape 2. Changez de modèle avec la commande `/model` puis tapez exactement le même prompt. Essayez au moins deux modèles différents parmi ceux disponibles.

Étape 3. Répondez par écrit à ces questions avant de continuer :

- Quelle différence observez-vous dans la longueur des réponses ?
- L'exemple Python est-il plus complet sur un modèle que sur l'autre ?
- Quel modèle vous semble le mieux calibré pour cette question simple ?

💡 Voir `../fondamentaux/guide-modeles-claude.md` pour le tableau complet Haiku / Sonnet / Opus / Fable.

Exercice 2 — Tâche complexe vs tâche simple

Contexte : Haiku coûte ~20x moins cher qu’Opus. La règle est de choisir le modèle minimal suffisant pour la tâche.

Étape 1. Tapez ce prompt (tâche simple) :

Donne-moi le nom de la capitale de l'Allemagne.

Étape 2. Tapez ce prompt (tâche complexe) :

J'ai une API REST qui retourne des données météo. Elle est parfois lente (> 3s).
Je veux implémenter un cache côté client avec une TTL de 10 minutes et un fallback sur les dernières données valides si l'API est hors ligne. Propose une architecture en Python avec les classes nécessaires, les types, et les cas limites à gérer.

Étape 3. Répondez :

- Pour la capitale de l’Allemagne, quel modèle auriez-vous dû utiliser ? Pourquoi ?
- Pour la question architecture, est-ce qu’un modèle léger suffit ou faut-il monter en gamme ?
- Formulez une règle personnelle en une phrase pour choisir votre modèle.

Exercice 3 — La fenêtre de contexte

Contexte : Claude ne “se souvient” pas : il relit toute la conversation à chaque message. Quand la conversation est trop longue, les premiers messages disparaissent.

Étape 1. Tapez ce message pour planter un marqueur :

NOTE IMPORTANTE : mon numéro de projet est PROJ-7734. Souviens-toi de ce numéro.

Étape 2. Tapez ensuite ce long message pour remplir le contexte :

Voici le code de mon module d'authentification. Analyse-le et explique chaque fonction :

```
import hashlib
import hmac
import secrets
import time
from typing import Optional

SECRET_KEY = "supersecret"
TOKEN_EXPIRY = 3600

def generate_token(user_id: str) -> str:
    timestamp = str(int(time.time()))
    payload = f"{user_id}:{timestamp}"
    signature = hmac.new(SECRET_KEY.encode(), payload.encode(), hashlib.sha256).hexdigest()
    return f"{payload}:{signature}"

def verify_token(token: str) -> Optional[str]:
    try:
        user_id, timestamp, signature = token.rsplit(":", 2)
        payload = f"{user_id}:{timestamp}"
        expected = hmac.new(SECRET_KEY.encode(), payload.encode(), hashlib.sha256).hexdigest()
        if not hmac.compare_digest(signature, expected):
            return None
        if int(time.time()) - int(timestamp) > TOKEN_EXPIRY:
            return None
        return user_id
    except Exception:
        return None

def generate_refresh_token() -> str:
    return secrets.token_urlsafe(32)

def hash_password(password: str) -> str:
    salt = secrets.token_hex(16)
    hashed = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(), 100000)
    return f"{salt}:{hashed.hex()}"

def verify_password(password: str, stored_hash: str) -> bool:
    salt, hashed = stored_hash.split(":", 1)
    new_hash = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(), 100000)
    return hmac.compare_digest(new_hash.hex(), hashed)
```

Étape 3. Après la réponse, tapez exactement :

Quel était mon numéro de projet ?

Étape 4. Répétez les étapes 1 et 2 plusieurs fois de suite (5 fois) pour allonger la conversation, puis retapez la question du numéro de projet.

Observez : à quel moment Claude ne retrouve plus le numéro PROJ-7734 ?

Étape 5. Répondez :

- Qu'est-ce que cela révèle sur la "mémoire" de Claude ?
- Comment le `CLAUDE.md` résout partiellement ce problème ? (Indice : il est relu à chaque session)

Exercice 4 — Estimation de coût

Contexte : Le coût est proportionnel aux tokens consommés. Voici les ordres de grandeur (chiffres illustratifs, vérifiez les prix actuels sur la page officielle Anthropic) :

Modèle	Input (par million tokens)	Output (par million tokens)
Haiku 4.5	~\$0.25	~\$1.25
Sonnet 4.6	~\$3	~\$15
Opus 4.8	~\$15	~\$75

Étape 1. Tapez ce prompt pour estimer la taille d'un échange type :

Compte le nombre approximatif de tokens dans ce message et dans ta réponse précédente.
Donne-moi juste deux chiffres : tokens input et tokens output.

Étape 2. Avec ces chiffres, calculez sur papier le coût de 500 requêtes similaires par jour pendant un mois :

- Sur Haiku : coût total ?
- Sur Sonnet : coût total ?
- Sur Opus : coût total ?

Étape 3. Répondez :

- Pour un chatbot interne qui répond à 500 questions/jour de support technique basique, quel modèle choisiriez-vous ? Justifiez en moins de 3 lignes.

Partie 2 — Premières entités

On configure l'environnement Claude Code pour un projet fictif : un assistant météo.

Exercice 5 — Votre premier **CLAUDE.md**

Contexte : Le `CLAUDE.md` est lu automatiquement au démarrage de chaque session. C'est le seul endroit où vous pouvez donner des instructions permanentes à Claude.

Étape 1. Dans votre dossier `~/atelier-claude`, créez le fichier `CLAUDE.md`. Tapez **exactement** ce prompt dans Claude Code :

Crée un fichier CLAUDE.md minimal (moins de 30 lignes) pour un projet Python appelé "meteo-agent". Ce projet :

- récupère la météo via une API externe
- ne modifie jamais de fichiers sans confirmation explicite de ma part
- utilise Python 3.11, black pour le formatage, pytest pour les tests
- a cette structure : `src/`, `tests/`, `scripts/`

Inclus une section "Règles absolues" avec 3 interdictions claires.

Étape 2. Vérifiez que le fichier créé fait moins de 30 lignes :

WC -1 CLAUDE.md

Étape 3. Fermez Claude Code (`/exit`) et rouvrez-le dans le même dossier. Tapez ce prompt pour vérifier que le [CLAUDE.md](#) est bien lu :

Quel est le nom de mon projet et quelles sont les règles absolues qui s'appliquent ici ?

Observez : Claude doit mentionner `meteo-agent` et les 3 interdictions sans que vous les ayez retapées.

Exercice 6 — Votre première skill

Contexte : Une skill est un fichier Markdown dans `.claude/commands/`. Claude l'utilise automatiquement quand le prompt ressemble à son déclencheur.

Étape 1. Créez le dossier des skills :

```
mkdir -p .claude/commands
```

Étape 2. Tapez ce prompt dans Claude Code pour générer le squelette de la skill :

Crée le fichier `.claude/commands/summarizing-weather.md` avec ce frontmatter exact :

```
---
name: summarizing-weather
description: >-
  Résume les conditions météo d'une ville sous forme de tableau structuré.
  À utiliser quand l'utilisateur demande la météo, les conditions actuelles,
  ou veut un résumé météo d'une ville.
allowed-tools: Read
---
```

Ajoute ensuite un corps de 10 lignes maximum avec :

- une section "## Workflow" avec 3 étapes numérotées
- une section "## Format de sortie" avec un tableau à 3 colonnes : Ville | Temp | Conditions

Étape 3. Testez le déclenchement avec ces 5 prompts (un par un) :

Quelle est la météo à Lyon ?

Donne-moi un résumé météo pour Bordeaux.

Il fait quel temps à Tokyo en ce moment ?

Calcule le carré de 17.

Rédige un email pour mon manager.

Observez : les 3 premiers doivent déclencher la skill (Claude mentionne son workflow ou suit son format). Les 2 derniers ne doivent PAS la déclencher.

Notez combien de prompts sur 5 ont bien déclenché ou non déclenché la skill.

Exercice 7 — Le `.claudeignore`

Contexte : Sans `.claudeignore`, Claude indexe TOUS les fichiers du projet au démarrage. Cela consomme des tokens inutilement et peut exposer des données sensibles.

Étape 1. Créez des fichiers de test :

```
mkdir -p data logs
echo "API_KEY=sk-secret-123" > .env
echo "export SECRET=abc" > .env.local
echo '{"city":"Paris","temp":18}' > data/cache.json
echo "[2026-01-01] Requête OK" > logs/app.log
echo "# Mon code" > src/main.py
```

Étape 2. Tapez ce prompt dans Claude Code :

```
Quels fichiers as-tu accès dans ce projet ? Liste-les tous.
```

Notez ce que Claude voit (il devrait voir `.env`, les logs, etc.).

Étape 3. Tapez ce prompt pour créer le `.claudeignore` :

```
Crée un fichier .claudeignore qui exclut :
- tous les fichiers .env et .env.*
- le dossier data/
- le dossier logs/
- les fichiers *.log
Ajoute un commentaire avant chaque groupe expliquant pourquoi on l'exclut.
```

Étape 4. Fermez et rouvrez Claude Code, puis retapez :

```
Quels fichiers as-tu accès dans ce projet ? Liste-les tous.
```

Observez : `.env`, `data/cache.json` et `logs/app.log` ne doivent plus apparaître. `src/main.py` et `CLAUDE.md` doivent toujours être visibles.

Exercice 8 — Choisir la bonne entité

Contexte : Skill, Hook, `CLAUDE.md` et MCP ont des rôles distincts. Le fichier `../choix_entite.md` contient la matrice de décision complète.

Étape 1. Lisez la section "Cheatsheet 6 questions" de `../choix_entite.md`.

Étape 2. Pour chaque besoin ci-dessous, déterminez quelle entité utiliser et notez votre réponse :

#	Besoin	Votre réponse
A	"Chaque fois que Claude écrit un fichier <code>.py</code> , lancer <code>black</code> automatiquement"	?
B	"Rappeler à Claude qu'il ne doit jamais utiliser <code>print()</code> dans ce projet"	?
C	"Permettre à Claude d'accéder à la base de données PostgreSQL de prod"	?
D	"Générer un rapport de tests formaté à la demande"	?
E	"Interdire toute commande <code>rm -rf</code> "	?

Étape 3. Tapez ce prompt pour valider vos réponses :

Pour chacun de ces 5 besoins, dis-moi quelle entité Claude Code utiliser (Skill, Hook, CLAUDE.md, MCP, ou settings.json) et pourquoi en une ligne :

- A. Lancer black automatiquement après chaque écriture de fichier .py
- B. Rappeler à Claude de ne jamais utiliser print() dans ce projet
- C. Permettre à Claude d'accéder à une base de données PostgreSQL
- D. Générer un rapport de tests formaté à la demande
- E. Interdire toute commande rm -rf

Comparez la réponse de Claude avec vos réponses de l'étape 2.

Partie 3 — Économie de tokens

Comprendre comment réduire les coûts sans réduire la qualité.

Exercice 9 — Impact du .claudeignore sur le contexte

Contexte : Chaque fichier indexé au démarrage consomme des tokens de contexte initial. Plus le projet est grand, plus c'est coûteux.

Étape 1. Créez un dossier volumineux artificiel :

```
for i in $(seq 1 20); do echo "# Module $i" > "src/module_$i.py"; done
```

Étape 2. Tapez ce prompt sans .claudeignore actif :

Combien de fichiers Python vois-tu dans ce projet ?

Étape 3. Ajoutez `src/` à votre `.claudeignore` :

```
echo -e "\n# Code source (trop volumineux pour ce test)\nsrc/" >> .claudeignore
```

Retapez le même prompt. **Observez** : Claude ne devrait plus voir les 20 modules.

Étape 4. Répondez :

- Dans un vrai projet avec 500 fichiers Python, quelle fraction du contexte initial économisez-vous en ignorant les dossiers de build et de dépendances (`node_modules/` , `__pycache__` / , `dist/`) ?

Exercice 10 — Divulcation progressive dans une skill

Contexte : Une skill trop longue est chargée entièrement dans le contexte à chaque invocation. La divulgation progressive déplace les détails longs dans un dossier `references/` .

Étape 1. Tapez ce prompt :

```
La skill .claude/commands/summarizing-weather.md est trop longue. Je veux la restructurer avec la divulgation progressive :  
1. Dans le SKILL.md, garde uniquement le frontmatter + un workflow de 5 lignes max  
2. Crée .claude/commands/references/weather-output-format.md avec le format de sortie détaillé (tableau avec exemples de 5 villes, icônes météo, règles de formatage)  
3. Dans le SKILL.md, référence ce fichier avec : "Voir references/weather-output-format.md"
```

Étape 2. Vérifiez la taille des deux fichiers :

```
wc -l .claude/commands/summarizing-weather.md  
wc -l .claude/commands/references/weather-output-format.md
```

Le `SKILL.md` principal doit faire moins de 20 lignes. Le fichier `references/` contient les détails.

Étape 3. Répondez :

- Si Claude charge la skill à 100 reprises dans la journée, combien de tokens économise-t-on si le `SKILL.md` passe de 80 lignes à 15 lignes ? (estimation approximative : 1 ligne $\approx$ 10 tokens)

Exercice 11 — `CLAUDE.md` concis vs verbeux

Contexte : Le `CLAUDE.md` est chargé à chaque message. Un `CLAUDE.md` de 300 lignes coûte des tokens à chaque tour de conversation, même quand son contenu n'est pas utile.

Étape 1. Tapez ce prompt pour créer une version verbeuse du `CLAUDE.md` :

```
Réécris mon CLAUDE.md en version "trop détaillée" : ajoute une section d'historique du projet (10 lignes), une section sur chaque convention de code (5 lignes chacune pour black, pytest, typing, docstrings, imports), et une section FAQ avec 5 questions-réponses. Total : environ 80 lignes. Sauvegarde-le dans CLAUDE-verbose.md (pas CLAUDE.md).
```

Étape 2. Comparez les tailles :

```
wc -l CLAUDE.md CLAUDE-verbose.md
```

Étape 3. Tapez ce prompt pour comprendre l'impact :

Si mon CLAUDE.md fait 80 lignes (environ 800 tokens) et que j'ai 50 échanges par session, combien de tokens supplémentaires sont consommés par rapport à un CLAUDE.md de 20 lignes ? Exprime le résultat en tokens et en coût approximatif avec Sonnet à \$3 par million de tokens input.

Étape 4. Répondez :

- Quelle information du [CLAUDE-verbose.md](#) mérite vraiment d'être là ?
- Quelle information appartient plutôt à un fichier de documentation lu à la demande ?

Exercice 12 — Le prompt caching (observation)

Contexte : Le prompt caching réutilise les calculs quand un préfixe de conversation est identique. Cela réduit le **coût** de facturation (pas le nombre de tokens traités).

Étape 1. Tapez ce prompt trois fois de suite sans modifier quoi que ce soit entre chaque :

Résume en 2 lignes ce que fait le fichier CLAUDE.md de ce projet.

Étape 2. Observez dans les logs de Claude Code (si disponibles) si la mention "cache" apparaît. Sinon, observez si la 2^e et 3^e réponse arrivent plus vite que la 1^{re}.

Étape 3. Modifiez légèrement le prompt et recommencez :

Résume en 3 lignes ce que fait le fichier CLAUDE.md de ce projet.

Étape 4. Répondez :

- Qu'est-ce qui doit être identique pour que le cache se déclenche ?
- Dans quel scénario réel le caching serait-il le plus utile : un chatbot avec beaucoup d'utilisateurs, ou un script qui tourne une fois par nuit ? Pourquoi ?

💡 Voir `../optimisation/guide-tokens.md` — section "Prompt caching" pour les détails techniques.

Livrable

À la fin du TP1, vous devez avoir dans `~/atelier-claude` :

```
~/atelier-claude/
├─ CLAUDE.md                ← créé à l'Ex5
├─ CLAUDE-verbose.md       ← créé à l'Ex11
├─ .claudeignore           ← créé aux Ex7 et Ex9
├─ .env                    ← créé à l'Ex7 (ignoré par .claudeignore)
├─ src/
│   └─ main.py
│   └─ module_1.py ... module_20.py
└─ .claude/
    └─ commands/
        └─ summarizing-weather.md ← créé à l'Ex6, restructuré à l'Ex10
        └─ references/
            └─ weather-output-format.md
```

Et un document personnel (texte libre) avec vos réponses aux questions de réflexion.